

DELTA TRACE: EFFICIENT SIGNED ATTRIBUTION FOR REASONING LANGUAGE MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

Explaining a reasoning model’s answer involves tracing how input evidence contributes to its prediction. These contributions carry both magnitude and direction: different sources can support or oppose the same response. We present DELTA TRACE, a method that propagates signed contributions from a response to its input sources. The method decomposes changes in model computations into content and routing contributions, follows them through attention and gated memory, and preserves their sum through a finite chain rule. An analytic treatment of attention normalization connects changes in routing probabilities to changes in attention scores. A corresponding memory recurrence tracks how evidence is retained, read, and updated. The resulting propagation admits tiled attention contractions and chunked memory computation, with sequence-linear auxiliary storage at fixed model dimensions. DELTA TRACE provides a common attribution framework for attention and hybrid reasoning models. Its evaluation protocol combines source recovery and the original FlashTrace faithfulness metrics with complete attribution cost measurements.

1 INTRODUCTION

Reasoning language models use input evidence across an extended sequence of computations. An explanation connects their answers to the sources that shape those answers. For a given response, a useful attribution identifies both the amount of contribution assigned to a source and its direction. Supporting and opposing evidence can coexist in the same context, and their contributions can cancel in the final prediction. Preserving this structure makes the explanation informative about how the model combines evidence.

Evidence enters a model’s prediction through its content and through the mechanisms that select and combine that content. Attention weights route value vectors, while memory gates determine how much stored information survives and how much new information is written. FlashTrace traces source importance for multi-token targets through span aggregation and recursive attribution (Pan et al., 2026). DELTA TRACE traces signed prediction contributions through these content and control computations. Its central question is how each source participates in the change of a specified response score.

DELTA TRACE explains a response by following the change from a reference input to the original input. Each model operation distributes its output change among the changes of its inputs. A reverse traversal composes these local distributions into a contribution for every input source. The same construction handles a small activation change and a transition across a nonlinear region: propagation uses the change across the two executions. This gives the method a direct connection to the response being explained and a common scale for supporting and opposing contributions.

The key design concerns interactions. In attention, DELTA TRACE assigns changing content through the routing used by the original input and retains routing changes as a separate contribution. An analytic finite softmax rule propagates the routing contribution to query and key scores. In gated memory, the same content-oriented construction follows decay, retrieval, and writing through the state recurrence. Shared normalization and feed-forward operators use fixed finite propagation rules. Their composition yields contribution conservation across the complete computation.

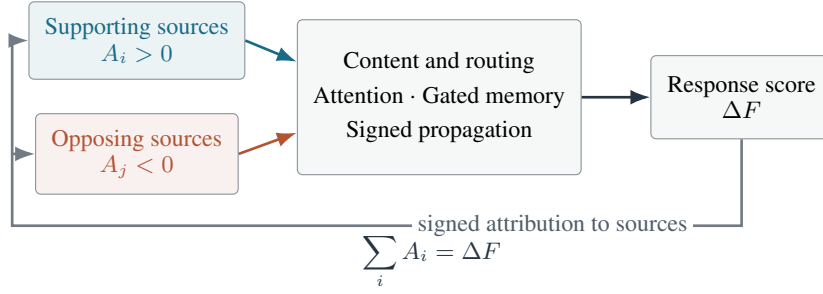


Figure 1: **Signed evidence tracing.** Input sources contribute through content, attention routing, and memory control. DELTATRACE propagates their signed contributions back from the response. Positive and negative contributions share a common target and sum to its change. The diagram depicts the method’s computation.

The propagation is organized around the model’s efficient computational primitives. Attention contributions are accumulated within tiles using the FlashAttention framework (Dao et al., 2022). Memory contributions reuse chunk-state propagation and batched contractions. The implementation retains vectors and chunk states while processing pairwise interactions within bounded tiles. This organization supports a common method across dense-attention and hybrid models, with model execution, attribution, and numerical precision specified separately.

This work develops three connected contributions. First, DELTATRACE traces signed response contributions through content and control with fixed rules shared across model architectures (Section 2). Second, analytic attention and memory rules connect this propagation to a finite chain rule and contribution conservation (Sections 2.3–2.4). Third, a tiled and chunked computation realizes these rules with sequence-linear auxiliary storage at fixed model dimensions (Section 3). The evaluation protocol links source recovery and faithfulness to complete attribution cost (Section 4).

2 DELTATRACE

2.1 SIGNED CONTRIBUTIONS TO A RESPONSE

The attribution target is the log-probability of a fixed response y under a context x ,

$$F(x; y) = \sum_{t \in \mathcal{T}} \log p_{\theta}(y_t \mid x, y_{<t}), \quad (1)$$

where $\mathcal{T}$ contains the selected response positions. The original input is x_1 and its reference is x_0 . In the evaluation protocol, the reference replaces eligible source tokens with EOS while retaining the response and its positions. Attribution is computed in the input embedding space. A source contribution A_i assigns part of $\Delta F = F(x_1; y) - F(x_0; y)$ to the embedding change at position i . Its sign specifies its direction within this decomposition.

2.2 CONTRIBUTION PROPAGATION

For an operation $v = f(u_1, \dots, u_k)$, DELTATRACE constructs linear maps D_j from the two executions such that

$$\Delta v = \sum_{j=1}^k D_j \Delta u_j. \quad (2)$$

An upstream contribution coefficient m_v propagates to input j as $D_j^{\top} m_v$. Contributions from multiple consumers add at their shared input. Starting with coefficient 1 at the scalar target, the reverse traversal produces m_i at input position i and returns

$$A_i = \langle m_i, x_{1,i} - x_{0,i} \rangle. \quad (3)$$

For a scalar nonlinearity f , the local coefficient is the divided difference $(f(u_1) - f(u_0))/(u_1 - u_0)$, with its derivative as the coincident-input limit. Linear transformations retain their original weights, and addition sends the coefficient to each summand.

Theorem 1 (Contribution conservation). *For an acyclic computation over real-valued operations satisfying Equation 2, reverse propagation from a scalar target yields*

$$\sum_i A_i = F(x_1; y) - F(x_0; y). \quad (4)$$

The result follows from the equality $\langle m_v, \Delta v \rangle = \sum_j \langle D_j^\top m_v, \Delta u_j \rangle$ at every operation. It places all source contributions on the same response scale and preserves cancellation as contributions meet along the computation graph. Appendix A gives the complete argument.

2.3 CONTENT AND ROUTING IN ATTENTION

For attention probabilities P and values V , the output is $Y = PV$. DELTATRACE decomposes its change as

$$\Delta Y = P_1 \Delta V + \Delta P V_0. \quad (5)$$

The first term assigns content changes through the attention used by the original input. The second assigns changes in routing their own contribution. The interaction $\Delta P \Delta V$ is included in the content term. With an upstream coefficient U , the corresponding coefficients are $m_V = P_1^\top U$ and $m_P = UV_0^\top$.

Attention probabilities are normalized scores. For a row $p_b = \text{softmax}(z_b)$, $b \in \{0, 1\}$, the logarithmic mean and its sum are

$$\ell_i = \frac{p_{1,i} - p_{0,i}}{\log p_{1,i} - \log p_{0,i}}, \quad \tau = \sum_i \ell_i, \quad (6)$$

with $\ell_i = p_{0,i}$ at coincident probabilities. All quantities are evaluated on the common unmasked support. The following identity connects routing changes to score changes:

Proposition 1 (Finite attention normalization). *For finite logits on a common support,*

$$\Delta p = \left(\text{diag}(\ell) - \frac{\ell \ell^\top}{\tau} \right) \Delta z. \quad (7)$$

The transpose action reduces to an elementwise product and a weighted row mean. For an upstream row u , its score coefficient is

$$m_z = \ell \odot \left(u - \frac{\ell^\top u}{\tau} \mathbf{1} \right). \quad (8)$$

The same logarithmic mean gives the response seed for log-softmax: $m_z = e_y - \ell/\tau$. Thus attention routing and the response target share an analytic normalization rule.

Query-key scores use the symmetric product identity,

$$\Delta(QK^\top) = \Delta Q \bar{K}^\top + \bar{Q} \Delta K^\top, \quad \bar{Q} = (Q_0 + Q_1)/2, \quad (9)$$

with $\bar{K}$ defined in the same way. The common operator rules appear in Table 1. They depend on the operator and its role, and are shared by the two model families. Normalization, rotary position transformations, and grouped heads follow their respective model definitions.

2.4 EVIDENCE IN GATED MEMORY

Gated delta memory combines retention with a content-dependent update (Yang et al., 2025). With column vectors and a key-by-value state $S_t \in \mathbb{R}^{d_k \times d_v}$, its per-token computation is

$$\begin{aligned} T_t &= \alpha_t S_{t-1}, & r_t &= T_t^\top k_t, \\ c_t &= v_t - r_t, & u_t &= \beta_t c_t, \\ S_t &= T_t + k_t u_t^\top, & o_t &= S_t^\top q_t. \end{aligned} \quad (10)$$

Table 1: Fixed propagation rules shared across architectures. Bars denote endpoint averages. Gated-memory operations extend the same content-oriented rule to recurrent states.

Operation	Change decomposition
Linear and residual	$\Delta(Wx + b) = W\Delta x; \Delta(a + b) = \Delta a + \Delta b$
Attention values	$\Delta(PV) = P_1\Delta V + \Delta PV_0$
Attention scores	$\Delta(QK^\top) = \Delta Q\bar{K}^\top + \bar{Q}\Delta K^\top$
SwiGLU	$\Delta(us) = \bar{s} \odot \Delta u + \bar{u} \odot \Delta s, s = \text{SiLU}(g)$
Output gating	$\Delta(ns) = s_1 \odot \Delta n + n_0 \odot \Delta s$
RMS normalization	Analytic finite rule for $wx/\sqrt{\text{mean}(x^2) + \epsilon}$

The query includes the model’s fixed attention scale. The state T_t contains retained evidence, r_t reads existing content, and u_t writes the update. DELTATRACE follows the change of this computation using fixed content-oriented products:

$$\begin{aligned}
\Delta T_t &= \alpha_{t,1}\Delta S_{t-1} + S_{t-1,0}\Delta\alpha_t, \\
\Delta r_t &= (\Delta T_t)^\top k_{t,1} + T_{t,0}^\top \Delta k_t, \\
\Delta u_t &= \beta_{t,1}(\Delta v_t - \Delta r_t) + c_{t,0}\Delta\beta_t, \\
\Delta S_t &= \Delta T_t + k_{t,1}\Delta u_t^\top + \Delta k_t u_{t,0}^\top, \\
\Delta o_t &= (\Delta S_t)^\top q_{t,1} + S_{t,0}^\top \Delta q_t.
\end{aligned} \tag{11}$$

This recurrence assigns contributions to new values, retrieval keys, queries, and both gates. Reversing it carries the response coefficient through stored evidence to earlier sources. Scalar gate nonlinearities and query/key normalization use the same finite propagation construction as the surrounding network.

3 EFFICIENT COMPUTATION

Tiled attention propagation. The attention rule requires three families of contractions: row reductions for the logarithmic mean, query/key coefficients from the score contribution, and value coefficients from the original-input probabilities. The implementation evaluates these contractions within the FlashAttention tiling framework (Dao et al., 2022). Native endpoint queries, keys, values, and log-normalizers determine the tile probabilities. The first sweep accumulates the row normalizers and weighted centers in Equation 8; subsequent sweeps accumulate query, key, and value coefficients. Endpoint averages are computed either in linear-size buffers or within the tiles. The persistent attention quantities are endpoint vectors, row statistics, and output coefficients.

Chunked memory propagation. Equation 11 induces a reverse state recurrence. DELTATRACE reuses the FLA implementation’s native chunk-state adjoints and combines them with the saved endpoint states through batched matrix products. A finite decay contraction is evaluated with an associative affine scan. The current implementation processes 64-token chunks, accumulating contributions to queries, keys, values, decay, and write gates. Convolution and surrounding projections use the installed model primitives and their linear transpose actions.

Storage and arithmetic. For sequence length T , fixed head dimension d , and fixed tile width C , attention coefficient vectors and row statistics occupy $O(Td)$ storage. Pairwise products reside in bounded tiles. Dense attention contractions retain $O(T^2d)$ arithmetic per head with a fixed number of sweeps. Chunked memory retains $O(Td + TC + (T/C)d^2)$ auxiliary storage per head: token coefficients, within-chunk products, and chunk-boundary states. Each expression is linear in T at fixed d and C . Native layer replay makes the same organization available across network depth, with checkpoint storage accounted for separately.

Native execution. The model runs its standard attention and memory backends. DELTATRACE consumes the two executions and applies separately defined contribution operators. Qwen3 uses the fixed FP16 implementation; Qwen3.5 uses BF16 attention and FLA with FP32 accumulation where supplied by the underlying operations. The complete cost includes endpoint execution, layer replay,

contribution propagation, and transfers. These components provide a common basis for latency, memory, and throughput measurements.

4 EVALUATION PROTOCOL

Source recovery and faithfulness. The evaluation uses the processed tasks and fixed generated trajectories from the authors’ `table1-data-v1` release accompanying FlashTrace (Pan et al., 2026).¹ The source-recovery protocol measures recall of gold evidence within the top 10% of eligible input tokens. The faithfulness protocol uses the authors’ 20-step deletion procedure and the released RISE and MAS scoring functions. Both metrics use the positive part of DELTATRACE’s source contributions, and the complete signed attribution is retained as a separate output. Recovery and faithfulness are reported alongside the signed evidence visualizations.

Aligned comparisons. The Qwen3-8B comparison uses the original input formatting, FP16 model loading, response targets, EOS handling, and eager scoring defined by the released experiment code. The authors’ `table1-data-v1` release supplies the reference results for complete released task selections, using one recursive hop for faithfulness and three hops for recovery. The Qwen3.5-9B comparison uses the same data and scoring construction and runs the fixed FlashTrace implementation on the same Qwen3.5 model. The DELTATRACE profiles share the rules in Table 1; gated memory uses Equation 11 at every corresponding layer.

Cost measurements. The measurement protocol records complete attribution latency, peak allocated memory, and throughput for real sample batches. Sequence length, model, dtype, and batch composition accompany each measurement. Loading, compilation, warmup, and measured execution are recorded separately. The cost breakdown follows the stages in Section 3, connecting the observed resource use to the propagation algorithm.

Mechanism comparisons. The central attention comparison fixes all other operators and evaluates the content-oriented identity in Equation 5 alongside symmetric and reverse-order product allocation. The same official examples and metrics connect the choice of interaction assignment to evidence recovery and response faithfulness. Signed source visualizations show how supporting and opposing contributions combine for the response target.

5 RELATED WORK

Prediction attribution. Integrated Gradients studies attribution through sensitivity and implementation invariance (Sundararajan et al., 2017). DeepLIFT propagates differences from reference activations and develops multiplier composition with a summation-to-delta property (Shrikumar et al., 2017). DELTATRACE develops finite propagation for normalized attention and gated memory, with explicit content and control allocations and a tiled implementation. These operator-level constructions connect its prediction target to the structure of contemporary reasoning models.

Reasoning and efficient sequence computation. FlashTrace combines span aggregation and recursive attribution to trace importance through reasoning sequences (Pan et al., 2026). Its released tasks, trajectories, and evaluation code provide the comparison protocol used here. FlashAttention organizes exact attention around tiled computation and memory traffic (Dao et al., 2022). Gated Delta Networks combine memory decay with delta-rule updates (Yang et al., 2025). DELTATRACE brings signed contribution propagation to both forms of sequence computation through their respective tiled and recurrent structures.

6 CONCLUSION

DELTATRACE traces signed contributions from a reasoning model’s response to its input evidence. Its fixed propagation rules connect content, attention routing, and gated memory to a common response change. Analytic attention normalization and finite state recurrence support contribution

¹<https://github.com/wbopan/flashtrace/releases/tag/table1-data-v1>

conservation, while tiled and chunked computation give the method sequence-linear auxiliary storage at fixed model dimensions. Together, these components establish a unified computational basis for signed evidence attribution in attention and hybrid language models.

AI USE STATEMENT

AI assistance supported conceptual development, mathematical derivations, research code, experimental design and analysis, literature retrieval, manuscript organization, and language editing. The manuscript uses the released evaluation data and model-generated trajectories specified in the evaluation protocol.

REFERENCES

- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *arXiv preprint arXiv:2205.14135*, 2022. URL <https://arxiv.org/abs/2205.14135>.
- Wenbo Pan, Zhichao Liu, Xianlong Wang, Haining Yu, and Xiaohua Jia. Towards long-horizon interpretability: Efficient and faithful multi-token attribution for reasoning LLMs. *arXiv preprint arXiv:2602.01914*, 2026. URL <https://arxiv.org/abs/2602.01914>.
- Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. Learning important features through propagating activation differences. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pp. 3145–3153. PMLR, 2017. URL <https://proceedings.mlr.press/v70/shrikumar17a.html>.
- Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pp. 3319–3328. PMLR, 2017. URL <https://proceedings.mlr.press/v70/sundararajan17a.html>.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2412.06464>.

A PROOFS AND OPERATOR IDENTITIES

A.1 CONTRIBUTION CONSERVATION

An acyclic computation can be expanded into elementary operations with explicit branches. At an operation $v = f(u_1, \dots, u_k)$, Equation 2 gives

$$\langle m_v, \Delta v \rangle = \sum_j \langle D_j^\top m_v, \Delta u_j \rangle. \quad (12)$$

Replacing a node's contribution by the contributions of its parents therefore preserves the total on a cut through the graph. At shared parents, coefficients from different consumers sum. Repeated substitution reaches the input cut and yields Equation 4, since the coefficient of the scalar target is 1. Constant inputs have zero difference and contribute zero to this sum.

A.2 FINITE ATTENTION NORMALIZATION

The common support contains positive probabilities. For endpoint b , $\log p_b = z_b - a_b \mathbf{1}$, where $a_b = \log \sum_i \exp(z_{b,i})$. The definition of the logarithmic mean gives

$$\Delta p = \ell \odot (\Delta z - \Delta a \mathbf{1}). \quad (13)$$

Probability normalization implies $\mathbf{1}^\top \Delta p = 0$, so

$$\Delta a = \frac{\ell^\top \Delta z}{\tau}. \quad (14)$$

Substitution proves Equation 7. For a target coordinate y ,

$$\Delta \log p_y = \left(e_y - \frac{\ell}{\tau} \right)^\top \Delta z. \quad (15)$$

The coincident-probability definition follows from the continuous limit of the logarithmic mean. The score operator is symmetric positive semidefinite: for $w_i = \ell_i / \tau$,

$$u^\top \left(\text{diag}(\ell) - \frac{\ell \ell^\top}{\tau} \right) u = \tau \left(\sum_i w_i u_i^2 - \left(\sum_i w_i u_i \right)^2 \right) \geq 0. \quad (16)$$

This is the weighted variance of u multiplied by τ .

A.3 PRODUCT AND NORMALIZATION RULES

For a bilinear map B , both

$$\Delta B(a, b) = B(\Delta a, \bar{b}) + B(\bar{a}, \Delta b), \quad (17)$$

$$\Delta B(a, b) = B(\Delta a, b_1) + B(a_0, \Delta b) \quad (18)$$

follow by expansion. Equation 5, the shared SwiGLU rule, and every product in Equation 11 are direct instances with the stated argument ordering. Subtraction and state addition are linear, completing the finite memory recurrence.

For RMS normalization $f(x) = w \odot x / r(x)$, $r(x) = \sqrt{\|x\|^2 / d + \epsilon}$, the symmetric product rule gives an analytic coefficient. With $\bar{x} = (x_0 + x_1) / 2$, $r_b = r(x_b)$, and $v = w \odot m$, its transpose action is

$$D_f^\top m = \frac{r_0^{-1} + r_1^{-1}}{2} v - \frac{2\bar{x} \langle \bar{x}, v \rangle}{d r_0 r_1 (r_0 + r_1)}. \quad (19)$$

The identity follows from $\Delta r = (2 \langle \bar{x}, \Delta x \rangle / d) / (r_0 + r_1)$ and $\Delta(r^{-1}) = -\Delta r / (r_0 r_1)$. The same construction covers fixed learned scale parameters and the query/key normalization used in the memory path.

B EVALUATION SPECIFICATION

The FlashTrace `table1-data-v1` release fixes the prompt transformation, eligible source positions, response target, gold mapping, and scoring implementation. DELTATRACE’s scalar target includes the released response and EOS. A source score enters the faithfulness function as $s_i = \max(A_i, 0)$; the released function determines deletion order and normalized contribution mass. Raw signed scores remain available for source visualizations. Full task comparisons use all released examples for the chosen task. Each result record identifies its model, method version, sample selection, metric implementation, and measured execution stages.